

CSCE 735

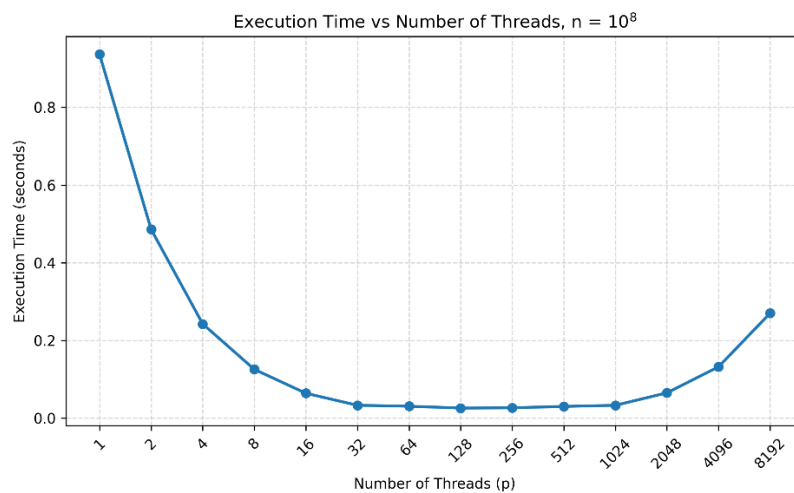
HW 1: Parallel Programming on a Multicore Multiprocessor

Name: Medha Venkata Subrahmanya Aditya Vellampalli

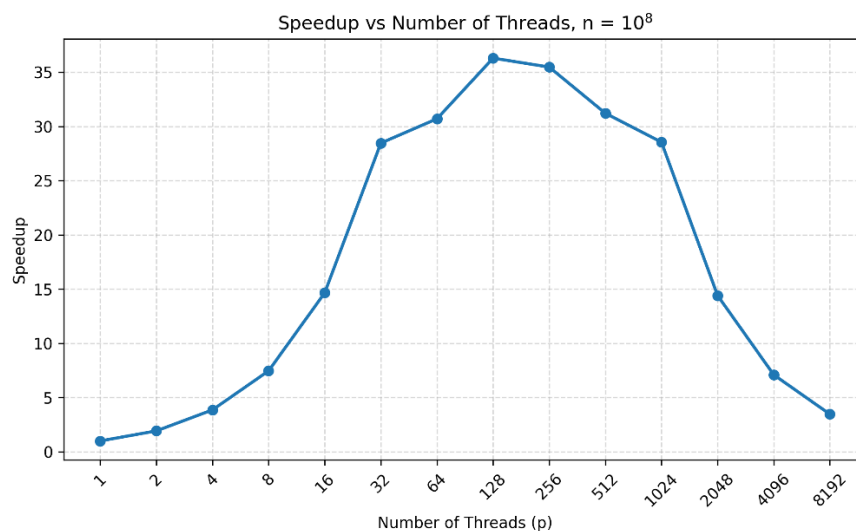
UIN: 237006700

Part 1. Shared-Memory Programming with Threads:

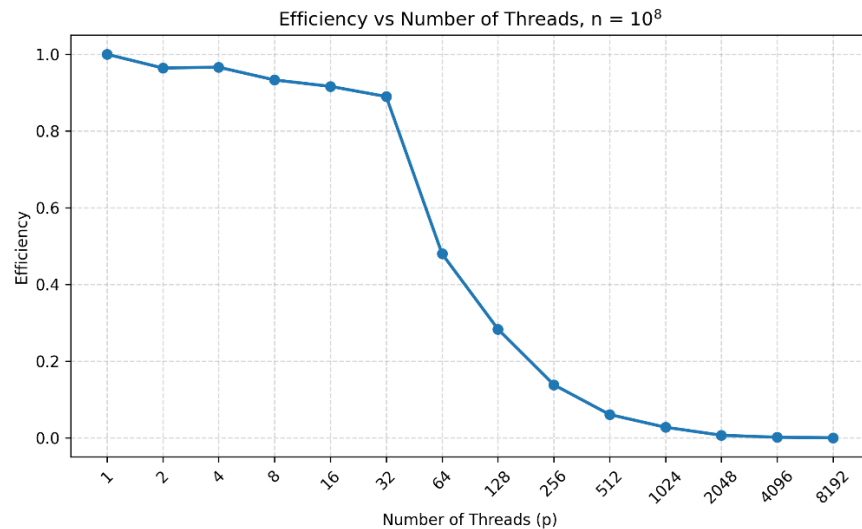
1. Execute the code for $n=10^8$ with p chosen to be $2k$, for $k = 0, 1, \dots, 13$. Using the experimental data obtained from these experiments, answer the following questions.
 1. For plots, use a logarithmic scale for the x-axis. 1.1. (10 points) Plot execution time versus p to demonstrate how time varies with the number of threads.



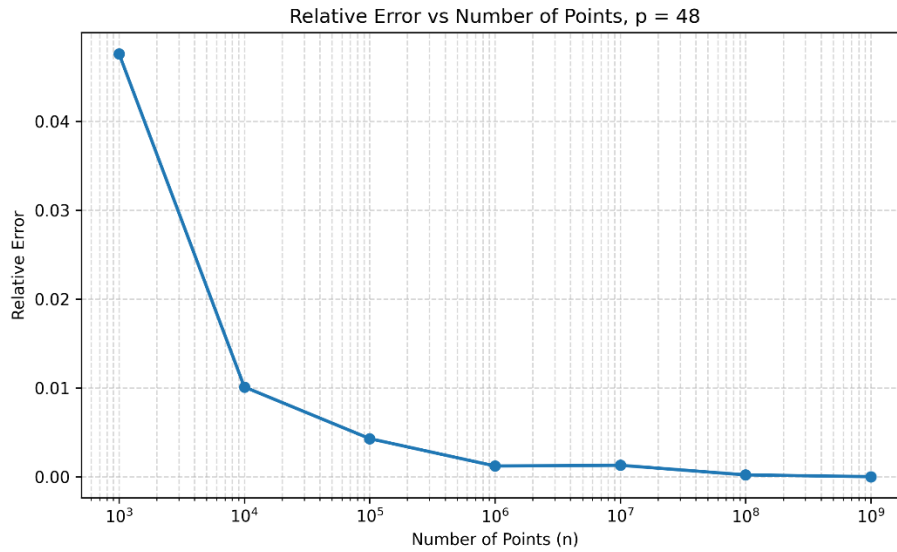
2. (10 points) Plot speedup versus p to demonstrate the change in speedup with p .



3. (5 points) Using the definition: $\text{efficiency} = \text{speedup}/p$, plot efficiency versus p to demonstrate how efficiency changes as the number of threads are increased.

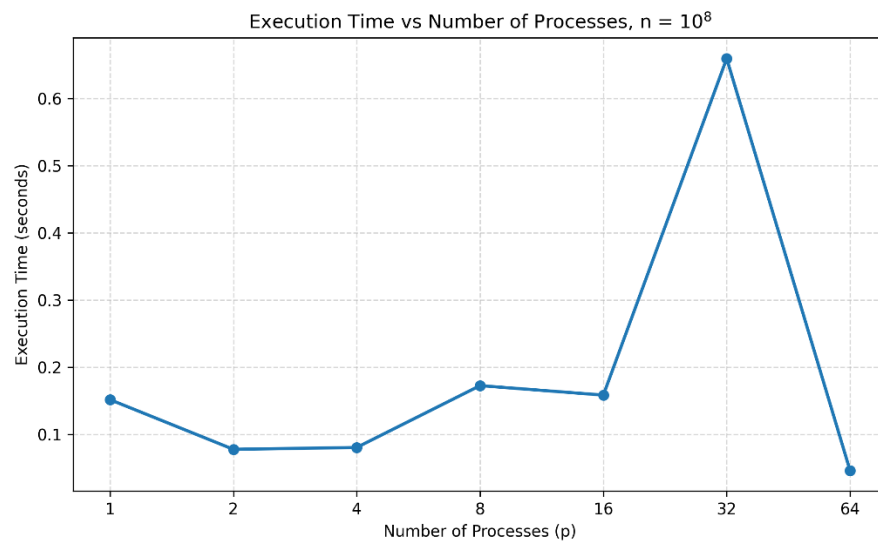


4. (5 points) In your experiments, what value of p minimizes the parallel runtime?
- For $n=10^8$, the minimum measured parallel runtime was 0.0258 s using $p=128$ threads.
2. Repeat the experiments with $n=10^{10}$ to obtain the execution time for $p=2^k$, for $k = 0, 1, \dots, 13$.
1. (5 points) In this case, what value of p minimizes the parallel runtime?
- For $n=10^{10}$, the minimum measured runtime was 2.2166s at $p = 512$ threads.
2. (5 points) Do you expect the runtime to increase as p is increased beyond a certain value? If so, why? And is this observed in your experiments.
- Yes. The runtime is expected to increase after the number of threads exceeds a certain value.
 - According to Amdahl's Law, adding more threads gives smaller performance gains because the serial part of the program cannot be parallelized. After a certain point, thread and synchronization overhead can outweigh the benefit.
 - For $n=10^{10}$, the minimum runtime of 2.2166 s occurs at 512 threads. The runtime then increases to 2.2232 s at 1024 threads, 2.2277 s at 2048 threads, 2.2622 s at 4096 threads, and 2.3616 s at 8192 threads.
3. (5 points) Do you expect that there would be a difference in the number of threads needed to obtain the minimum execution time for two values of n ? Is this observed in your experiments.
- Yes. A larger problem can benefit from more threads because there is more computation to divide before parallel overhead becomes significant.
 - This is observed in the experiments. For $n=10^8$, the minimum runtime occurred at 128 threads, while for $n=10^{10}$, it occurred at 512 threads. So, the larger problem required more threads to achieve the minimum execution time.
4. (5 points) Plot error versus n to illustrate accuracy of the algorithm as a function of n . You may have to run experiments with different values of n ; for example n could be chosen to be 10^k , for $k = 3, \dots, 9$. Use $p = 48$.

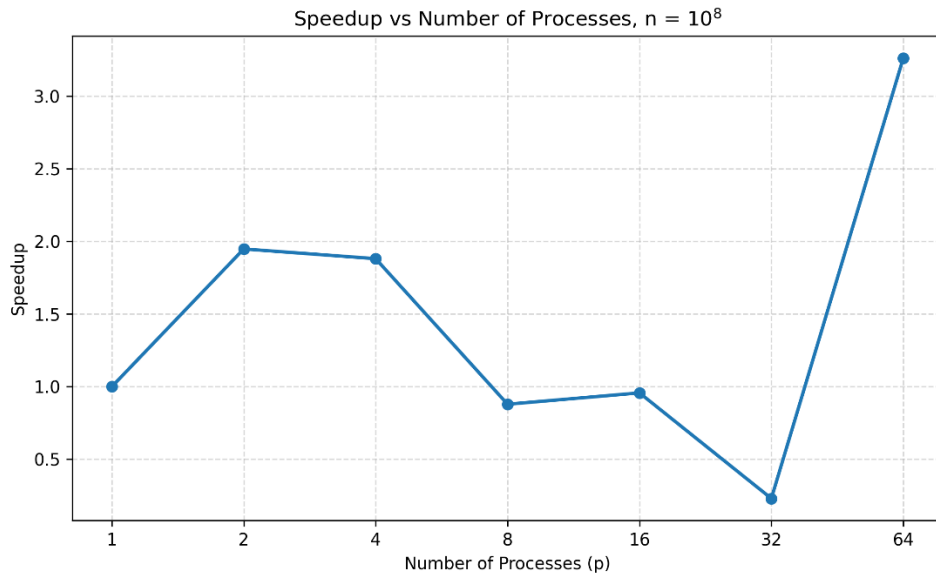


Part 2. Distributed-Memory Programming with MPI:

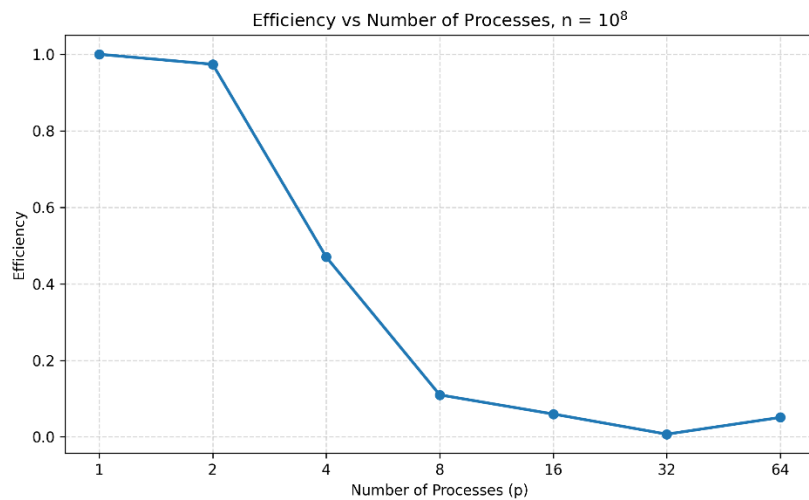
5. Execute the code for $n=10^8$ with p chosen to be $2k$, for $k = 0, 1, \dots, 6$. Specify $\text{ntasks-per node}=4$ in the job file. Using the experimental data obtained from these experiments, answer the following questions. For plots, use a logarithmic scale for the x-axis.
 1. (10 points) Plot execution time versus p to demonstrate how time varies with the number of processes.



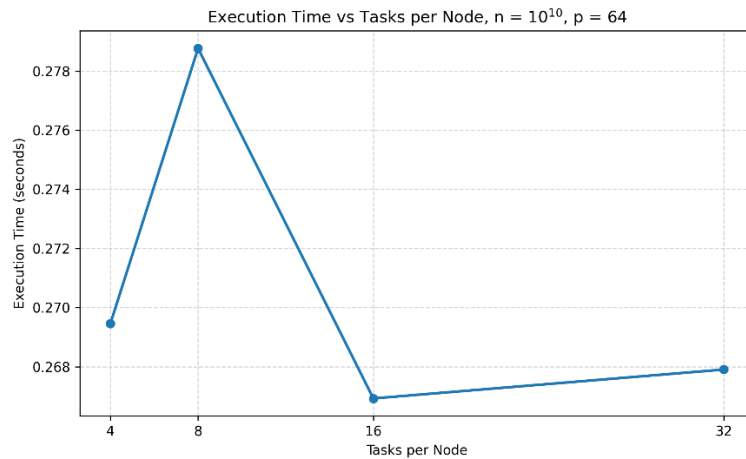
2. (10 points) Plot speedup versus p to demonstrate the change in speedup with p .



3. (5 points) Using the definition: $\text{efficiency} = \text{speedup}/p$, plot efficiency versus p to demonstrate how efficiency changes as the number of processes is increased.

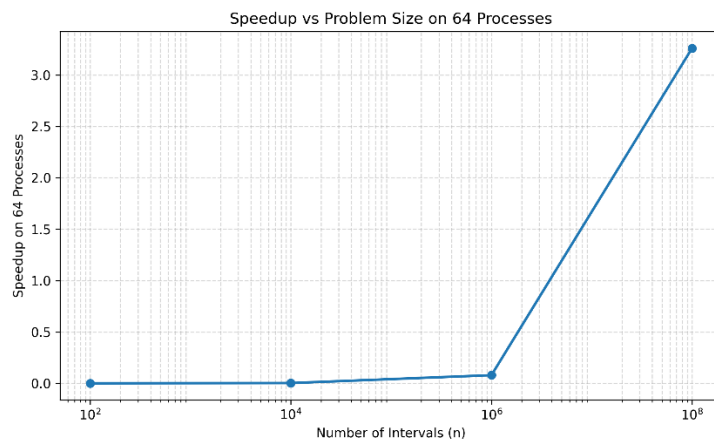


4. (5 points) What value of p minimizes the parallel runtime?
- The minimum measured parallel runtime was 0.04658476 s using $p=64$ processes.
6. (10 points) With $n=10^{10}$ and $p=64$, determine the value of ntasks-per-node that minimizes the total_time . Plot time versus ntasks-per-node to illustrate your experimental results for this question.



The minimum measured total time was 0.26692614 s with ntasks-per-node = 16. Therefore, among the tested configurations, ntasks-per-node = 16 minimizes the execution time.

7. Execute the code with $p=64$ for $n=10^2$, 10^4 , 10^6 and 10^8 , with ntasks-per-node=4.
 1. (5 points) Plot the speedup observed as a function of n on $p=64$ w.r.t. $p=1$. You will need to obtain execution time on $p=1$ for $n=10^2$, 10^4 , 10^6 and 10^8 .



2. (5 points) Plot the relative error versus n to illustrate the accuracy of the algorithm as a function of n .

